LAB

4

Operating Systems · FCIM / FAF · UTM · 2026-2027

The xv6 Shell from the Outside

B **Track B: build your own small OS (xv6) · 90 minutes**

AT A GLANCE

Part 1	Fix sleep for the current xv6	~15 min
Part 2	bash versus the xv6 shell	~30 min
Part 3	Who expands *? A program that prints its argv	~25 min
Part 4	Read the shell	~15 min
You submit	user/sleep.c (working on your tree), user/args.c, args_linux.c, and the Makefile with your UPROGS lines.	

Goal

The whole user interface of xv6 is one program, user/sh.c, in 499 lines. Today you use it side by side with bash and discover, by experiment, what a shell must do and what bash adds on top. You also write a tiny program that prints the arguments it receives, which shows exactly who expands * and who removes quotes. First, you make sure your sleep program from Lab 2 builds on the current xv6.

Before you start

Check that xv6 builds and boots on your laptop. If the check fails, run the script without --check (5 to 10 minutes, it installs everything and downloads xv6 into ~/xv6-riscv; your existing work there is never touched):

TERMINAL

```
bash os2026-setup.sh --check
bash os2026-setup.sh            # only if the check failed
```

PART 1 Fix sleep for the current xv6 ~15 min

In August 2025 the xv6 authors renamed the system call sleep() to pause(). Check which name your tree uses:

TERMINAL

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

If user.h declares pause, change the call in your user/sleep.c (and make sure $U/_sleep\ is in UPROGS):

FILE user/sleep.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
```

```
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));     // was sleep() before Aug 2025
  exit(0);
}
```

TERMINAL

```
make qemu
$ sleep 20
$ sleep
usage: sleep <ticks>
```

Makefile reminder

Each UPROGS line starts with a TAB and ends with a backslash. Nothing may follow the backslash, not even a comment. Exit QEMU with Ctrl-A then X.

PART 2 bash versus the xv6 shell ~30 min

Run every line below twice: once in bash on Linux (in an empty test folder that contains a file README), once at the xv6 $ prompt. Fill in the table in notes.md: what each shell printed, and why.

Command	Tests	bash	xv6
echo hello > f ; cat < f	redirection		
ls | grep READ	pipe		
(echo a; echo b) | wc	grouping, list		
echo one ; echo two	sequence		
echo bg &	background		
ls *	globbing		
echo $HOME	variables		
echo "a b"	quotes		
mkdir d ; cd d ; ls ..	PATH		
arrow up, Tab, Ctrl+R	history, completion		

Observe (write in notes.md)

1. List the features both shells have, and the ones only bash has.
2. In xv6, ls stops working after cd d. Why? Find a way to run ls from inside d anyway. (Hint: where do the xv6 programs live?)
3. Which missing feature would you miss most in daily work, and which would be easiest to add?

PART 3 Who expands *? A program that prints its argv ~25 min

Write user/args.c for xv6 and add $U/_args\ to UPROGS:

FILE user/args.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

Write the same program for Linux (args_linux.c: include <stdio.h> instead of the two xv6 headers, return 0 instead of exit(0)), compile it with gcc, and run the SAME lines in both shells:

TERMINAL

```
args one   two  three
args *.c "x y"
args $HOME
args 'single quoted'
```

Observe (write in notes.md)

1. Paste both outputs for `args *.c "x y"`. How many arguments did each program receive?
2. So who turns *.c into file names and removes the quotes: the program, the kernel, or the shell?
3. What happened to the extra spaces in `one   two  three`?

PART 4 Read the shell ~15 min

TERMINAL

```
wc -l user/sh.c
grep -n "#define" user/sh.c        # the kinds of commands
grep -n "case " user/sh.c          # where each kind is run
grep -n "cd" user/sh.c | head
```

Observe (write in notes.md)

1. Name the five kinds of command the xv6 shell knows and the character that produces each.
2. Where is cd handled, and why is it not run as a separate program?
3. Search sh.c for any code that handles * or $. What does that explain about Part 2?

FINAL Wrap-up and deliverable ~5 min

You submit:

- ☐ user/sleep.c (working on your tree), user/args.c, args_linux.c, and the Makefile with your UPROGS lines.
- ☐ notes.md: the comparison table and the answers to the "Observe" boxes.

Grading (out of 10):

Criterion	Points
sleep builds and works on the current tree	**2**
Comparison table complete and explained	**3**
args on both systems, the expansion question answered	**3**
Reading sh.c: command kinds, cd, why no globbing	**2**
Total	**10**